

INFO-F-201 — Systèmes d'exploitation

Introduction au C

Joël Goossens (N8.107) Olivier Markowitch (N8.106)

Arnaud Leponce (N8.213) Attilio Discepoli (N8.210)

Alexis Reynouard (N8.215)

1 Pourquoi le C ?

1.1 Un programme en langage machine

Imaginez que nous voulons construire un processeur simple avec trois espaces mémoires, appelés *gauche*, *droite* et *résultat*. On peut connecter *gauche* et *droite* à un circuit électronique qui fait une addition, et stocker ce résultat dans *résultat*. Pour ajouter plus de possibilités, on connecte aussi un circuit de soustraction à *gauche* et *droite*, puis un *sélecteur* qui choisit, selon une commande reçue, quel résultat utiliser stocker dans *résultat*. Par exemple: 1 demande de stocker le résultat de l'addition et 2 celui de la soustraction. Ces numéros sont appelées opcodes (codes d'opération). En écrivant une suite d'opcodes (convertis en binaire), on crée un programme exécutable par la machine.

1.2 L'assembleur: rendre les programmes lisibles

Pour faciliter la programmation, au lieu d'écrire directement ces nombres binaires, on utilise un langage appelé assembleur. On remplace les opcodes par des noms symboliques, par exemple «ADD» pour addition. Chaque processeur a son propre langage assembleur qui correspond aux instructions qu'il comprend. L'assembleur est simplement une couche lisible sur le code machine. Les premiers «compilateurs» assembleur vers code machine étaient des sortes de machine à écrire. Sur le clavier, les touches correspondaient à des opérations. Plutôt que de déposer de l'encre sur une feuille, la machine faisait des trous dans une carte perforée. Ces trous étaient l'opcode en binaire de l'opération voulue.

1.3 L'évolution des langages de programmation

De nombreux autres langages sont ensuite apparus, souvent spécialisés pour un matériel ou un domaine (mathématiques, physique, etc.). Ces langages étaient trop proches du matériel ou trop abstraits, et ne permettaient pas toujours de manipuler les données et les variables avec différentes sémantiques (valeur, référence, alias, pointeur) facilement. En 1969, le langage B a été conçu pour combiner un bon contrôle machine avec un certain niveau d'abstraction. Puis, en 1972, Dennis Ritchie, un des deux ingénieurs de B, a créé le langage C pour réécrire le système d'exploitation Unix, alliant *lisibilité*, *efficacité* et *portabilité*.

1.4 Les forces du langage C

Le langage C adopte un juste niveau d'abstraction: il permet de contrôler précisément la machine tout en offrant un code clair et efficace, utilisable sur différentes architectures. Il offre plusieurs avantages clés :

Sémantique flexible des variables On peut copier une variable (sémantique de valeur) ou prendre son adresse (sémantique de pointeur), permettant ainsi plusieurs noms pour une même donnée.)

Types explicites Chaque variable a un type qui détermine sa taille en mémoire et comment interpréter ses bits. Cela permet au compilateur de choisir l'instruction machine adaptée pour effectuer les opérations, ce qui n'existait pas dans B.

Création de types composés Possibilité de définir des types complexes par agrégation.

Portabilité efficace La taille des types principaux n'est pas rigide, ce qui permet au même code d'être compilé efficacement sur plusieurs machines.

1.5 Le C et les systèmes d'exploitation

C est donc idéal écrire un système d'exploitation (OS). Mais il l'est aussi pour communiquer avec l'OS.

Un OS rend le matériel accessibles via des fonctions appelées appels système. Ces appels sont décrits précisément en termes de mémoire, indépendamment du langage, mais leur utilisation est aisée en C. En pratique, faire un appel système en C revient souvent à appeler une petite fonction qui s'occupe de communiquer avec le noyau de l'OS.

Le code en C est souvent très proche du langage assembleur, ce qui permet d'écrire des couches logicielles très performantes et proches du matériel, tout en restant plus lisible et maintenable que l'assembleur pur. C'est pourquoi la majorité des noyaux modernes comme Unix, Linux, macOS ou Windows sont écrits principalement en C.

2 Variables, adresses et pointeurs

2.1 Sémantique par défaut des variables

En C, lorsqu'on déclare une variable, elle contient une valeur. Par défaut, la sémantique des variables est une *sémantique de copie* : cela signifie que quand on affecte une variable à une autre, ou quand on passe une variable en argument à une fonction, la valeur est copiée.

```
int a = 5;
int b = a; // ici la valeur 5 est copiée dans b
b = 10;    // ceci ne modifie pas a
```

LISTING 1 - EXEMPLE DE COPIE DE VARIABLE

Ainsi, modifier la variable `b` n'a pas d'effet sur `a` puisque ce sont deux emplacements mémoire différents.

2.2 Qu'est-ce qu'une adresse ?

Une *adresse* est un numéro unique qui identifie un emplacement en mémoire. Chaque variable occupe un certain emplacement mémoire, et on peut accéder à cet emplacement via son adresse.

On peut obtenir l'adresse d'une variable en utilisant l'opérateur `&`. On peut aussi parler de ce qui est à une adresse avec l'opérateur `*`.

```
int x = 3;
printf("Adresse de x : %p\n", (void*)&x);
*(&x) = 4; // équivalent à "x = 4"
```

LISTING 2 - OBTENIR L'ADRESSE D'UN VARIABLE

2.3 Pointeurs

Un *pointeur* est le type d'une variable qui stocke une *adresse* mémoire. Ou plutôt, une variable qui stocke l'adresse d'un entier doit être de type « pointeur vers un entier ». Ceci s'écrit: `int*`. « Pointeur » et « adresse » sont donc synonyme.

Quand l'étoile `*` est utilisée après un **type** `X`, elle signifie: « pointeur vers `X` » / « adresse d'un `X` ». C'est la déclaration d'un type. Ce n'est pas une opération. Par exemple, pointeur vers `int`: `int*`.

Mais quand l'étoile est utilisée avant une **adresse** `Y`, elle signifie: « ce qui se trouve à l'adresse `Y` ». (On l'appelle, opérateur de déréférencement.) C'est ce qu'on a fait dans l'exemple précédent. C'est une opération. Donc, devant un pointeur (qui est une adresse), l'étoile permet d'accéder à la valeur pointée.

```
int y = 7;
int *ptr = &y; // ptr est une variable de type pointeur vers entier
printf("%d\n", *ptr); // affiche la valeur de y (7) via son pointeur
```

LISTING 3 - DÉCLARATION ET UTILISATION D'UN POINTEUR

Exercice 1. Partez du fichier `exo1.c` qui se trouve sur l'UV. Compilez avec `make exo1`.

- Déclarez deux variables entières. Affectez la valeur de l'une dans l'autre. Modifiez la deuxième variable et expliquez pourquoi la première n'est pas modifiée.
- Affichez l'adresse de chacune de ces variables.
- Déclarez un pointeur vers une des variables et affichez la valeur de la variable à travers le pointeur.

Réponse : Retrouvez la solution sur l'UV.

Dans cet exemple, on voit que la variable `b` est indépendante de `a`, car la valeur est copiée. Les adresses imprimées montrent que `a` et `b` ne partagent pas le même emplacement mémoire. Enfin, le pointeur `ptr` contient l'adresse de `a`, et le déréférencement `*ptr` permet d'accéder à sa valeur.

3 Structure de la mémoire

Lorsque vous utilisez des variables en C, leur contenu peut se trouver dans un de ces deux endroits de la mémoire: le *heap* ou le *stack*.

3.1 Le stack

Conceptuellement, le *stack*, le *call stack* ou la pile (d'exécution) en français est la structure de données qui enregistre les informations au sujet des fonctions appelées successivement. On y trouve, entre autres, les variables locales des fonctions. Chaque appel de fonction donne lieu à un nouveau *stack frame* sur le dessus de la pile et, inversement, chaque retour de fonction donne lieu

à une suppression du *stack frame* sur le dessus de la pile. Ce procédé est illustré par du code dans le Listing 4 et visuellement dans la Fig. 1.

```
int main() {
    int x = 3;
    int r = foo();
    return 0;
}

int foo() {
    int y = 7;
    return y - 2;
}
```

LISTING 4 - EXEMPLE D'APPEL DE FONCTION.

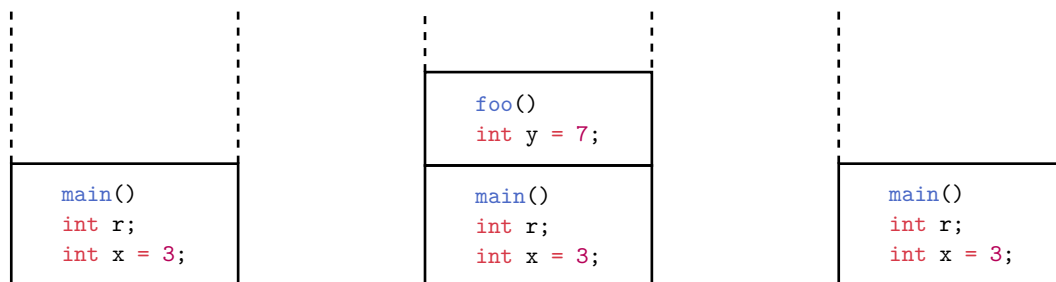


FIG. 1 - ÉVOLUTION DE LA PILE LORS D'APPELS ET RETOURS DE FONCTION.

Très concrètement, le *stack* est une zone de la mémoire dans laquelle réside la structure de données décrite plus haut. Étant donné que le *stack* enregistre les appels successifs de fonction, il est important de noter que chaque *thread* dispose de son propre *stack*.

Une conséquence de la manière dont le *stack* fonctionne est que les variables qui s'y trouvent ne peuvent pas changer de taille. En effet, si une variable se trouvant dans le `main` devait grandir (en nombre de bytes), il faudrait déplacer toutes les variables qui se trouvent *au dessus*, ce qui serait très coûteux. Plus encore: la taille des variables, c'est-à-dire le nombre de bytes qu'elles occupent en mémoire, est connu au moment de la compilation.

3.2 Le heap

Le *heap* est lui aussi une zone en mémoire dans laquelle on peut stocker des variables dont la taille peut varier au cours du temps. Le *heap* est géré par le système d'exploitation et il est possible de lui demander de nous attribuer de la mémoire à la demande. Cette zone mémoire est attachée à un pointeur vers la zone attribuée.

Contrairement au *stack*, le *heap* est spécifique au processus et est partagé entre ses différents threads.

3.2.1 Allocation de mémoire dynamique

Dans le langage C, cette demande se fait à l'aide de la fonction `malloc` comme indiqué dans le Listing ref{lst:malloc}. Notez qu'il existe toute une famille de fonctions d'allocation de mémoire telles que `calloc` ou `realloc`. Consultez le manuel pour plus d'informations (`man malloc`).

```

int taille;
scanf("%d", &taille);
int* tableau = (int*) malloc(sizeof(int) * taille);
if (tableau == NULL) {
    // Si malloc échoue, il retourne 0 / NULL,
    // et change la valeur de la variable globale errno pour indiquer l'erreur.
    // La fonction perror utilise errno pour indiquer l'erreur.
    perror("L'allocation de la mémoire a échoué")
    // Sur un système en anglais, la ligne ci-dessus affiche:
    // L'allocation de la mémoire a échoué: Cannot allocate memory
    exit(1);
}

```

LISTING 5 - ALLOCATION DYNAMIQUE AVEC MALLOC

Remarquez qu'il est important de vérifier la validité du pointeur alloué! Si jamais l'OS renvoie NULL, cela signifie probablement qu'il n'a plus de mémoire disponible, auquel cas il est urgent de terminer le programme.

Exercice 2. *Tous les pointeurs valides pointent-ils vers le heap ? Expliquez ou donnez un contre-exemple.*

Réponse : Non. On peut prendre l'adresse d'une variable du stack en faisant `int *p = &x;`.

Exercice 3. *Sachant que sur un système à 64 bits, une adresse tient sur 64 bits. Que renvoient `sizeof(int *)`, `sizeof(char *)` et `sizeof(int **)` ?*

Réponse : Dans tous les cas, une adresse fait 64 bits (donc 8 bytes). La réponse est donc systématiquement 8.

3.2.2 Libération de mémoire dynamique

Notez que dans le langage C, la mémoire allouée dynamiquement n'est pas libérée automatiquement, c'est-à-dire qu'il n'y a pas de *garbage collector*. Par conséquent, tant que le programme tourne, la mémoire allouée n'est pas libérée, ce qui peut causer des fuites de mémoire, ou *memory leaks*. Lorsqu'une zone mémoire n'est plus utilisée, il faut appeler la fonction `free()` qui libère la mémoire allouée.

```

int* tableau = (int*) malloc(sizeof(int) * taille);
if (tableau == NULL) {
    exit(1);
}
/* Utilisation du tableau */
// ...
/* Le tableau n'est plus utilisé */
free(tableau);

```

LISTING 6 - LIBÉRATION DE MÉMOIRE AVEC FREE

Exercice 4. *Récupérez le fichier header `list.h` sur l'UV et écrivez un fichier `list.c` qui permet de gérer une liste dynamique d'entiers. Vous devez écrire le corps des fonctions déclarées dans `list.h`. Ensuite, écrivez un `main()` qui utilise ces fonctionnalités.*

Réponse : Retrouvez les solutions sur l'UV dans un fichier zip.